МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность СУИ

Выполнил:
Кульбеда А. Ю.
студент группы АС-68

Проверил:
Крощенко А. А.
ст. преп. кафедры ИИТ,

«___» _____ 2026г.

Брест 2026

Цель работы: На практике сравнить работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Вариант 4

1. `Digits`
2. Распознать, какая цифра (от 0 до 9) изображена на картинке 8x8 пикселей
3. Задания:
    - Загрузите встроенный `scikit-learn` набор данных `digits`;
    - Разделите данные на обучающую и тестовую выборки;
    - Обучите три модели (k-NN, Decision Tree, SVM) для многоклассовой классификации;
    - Для каждой модели выведите `classification_report` (sklearn.metrics), содержащий основные метрики для каждого класса;
    - Сравните общую точность моделей и определите, какая из них лучше всего подходит для этой задачи.

Код программы:

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# загружаем датасет digits
digits = load_digits()
X = digits.data
y = digits.target

print("Размер данных:", X.shape)
print("Классы:", sorted(int(val) for val in set(y)))

# делим на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("Обучающая:", X_train.shape[0], "Тестовая:", X_test.shape[0])

# подбираем k для k-NN
print("\nПодбор k для k-NN:")
best_k = 1
best_acc = 0
for k in range(1, 21):
    knn = KNeighborsClassifier(n_neighbors=k)
```

```
    knn.fit(X_train, y_train)
    pred = knn.predict(X_test)
    acc = accuracy_score(y_test, pred)
    print("k =", k, "accuracy =", round(acc, 4))
    if acc > best_acc:
        best_acc = acc
        best_k = k
print("Лучшее k =", best_k, "accuracy =", round(best_acc, 4))

# обучаем три модели
models = {
    "k-NN": KNeighborsClassifier(n_neighbors=best_k),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "SVM": SVC(random_state=42)
}

results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    acc = accuracy_score(y_test, pred)
    results[name] = acc
    print("\nМодель:", name)
    print("accuracy =", round(acc, 4))
    print(classification_report(y_test, pred, digits=4,
zero_division=0))

# сравниваем модели и выводим победителя
print("\nСравнение моделей:")
for name, acc in results.items():
    print(name, "->", round(acc, 4))

best_model = max(results, key=results.get)
print("\nЛучшая модель:", best_model, "с точностью",
round(results[best_model], 4))
```

Вывод программы:

```
Размер данных: (1797, 64)
Классы: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
Обучающая: 1437 Тестовая: 360

Подбор k для k-NN:
k = 1 accuracy = 0.9861
k = 2 accuracy = 0.9861
k = 3 accuracy = 0.9861
k = 4 accuracy = 0.9833
k = 5 accuracy = 0.9833
k = 6 accuracy = 0.9806
k = 7 accuracy = 0.9833
k = 8 accuracy = 0.9778
k = 9 accuracy = 0.9806
k = 10 accuracy = 0.9778
k = 11 accuracy = 0.9778
k = 12 accuracy = 0.975
k = 13 accuracy = 0.975
k = 14 accuracy = 0.9778
k = 15 accuracy = 0.975
k = 16 accuracy = 0.9778
k = 17 accuracy = 0.9694
k = 18 accuracy = 0.9694
k = 19 accuracy = 0.9667
k = 20 accuracy = 0.9667
Лучшее k = 1 accuracy = 0.9861
```

```
Модель: k-NN
accuracy = 0.9861
              precision    recall  f1-score   support

           0     1.0000    1.0000    1.0000        36
           1     0.9231    1.0000    0.9600        36
           2     1.0000    1.0000    1.0000        35
           3     1.0000    1.0000    1.0000        37
           4     0.9730    1.0000    0.9863        36
           5     1.0000    1.0000    1.0000        37
           6     1.0000    1.0000    1.0000        36
           7     1.0000    1.0000    1.0000        36
           8     0.9697    0.9143    0.9412        35
           9     1.0000    0.9444    0.9714        36

    accuracy                         0.9861       360
   macro avg     0.9866    0.9859    0.9859       360
weighted avg     0.9867    0.9861    0.9861       360
```

```
Модель: Decision Tree
accuracy = 0.825
              precision    recall  f1-score   support

           0     0.9211    0.9722    0.9459        36
           1     0.8387    0.7222    0.7761        36
           2     0.7778    0.8000    0.7887        35
           3     0.7895    0.8108    0.8000        37
           4     0.8611    0.8611    0.8611        36
           5     0.8750    0.9459    0.9091        37
           6     0.8108    0.8333    0.8219        36
           7     0.8378    0.8611    0.8493        36
           8     0.7059    0.6857    0.6957        35
           9     0.8182    0.7500    0.7826        36

    accuracy                         0.8250       360
   macro avg     0.8236    0.8242    0.8230       360
weighted avg     0.8241    0.8250    0.8237       360
```

```
Модель: SVM
accuracy = 0.9917
              precision    recall  f1-score   support

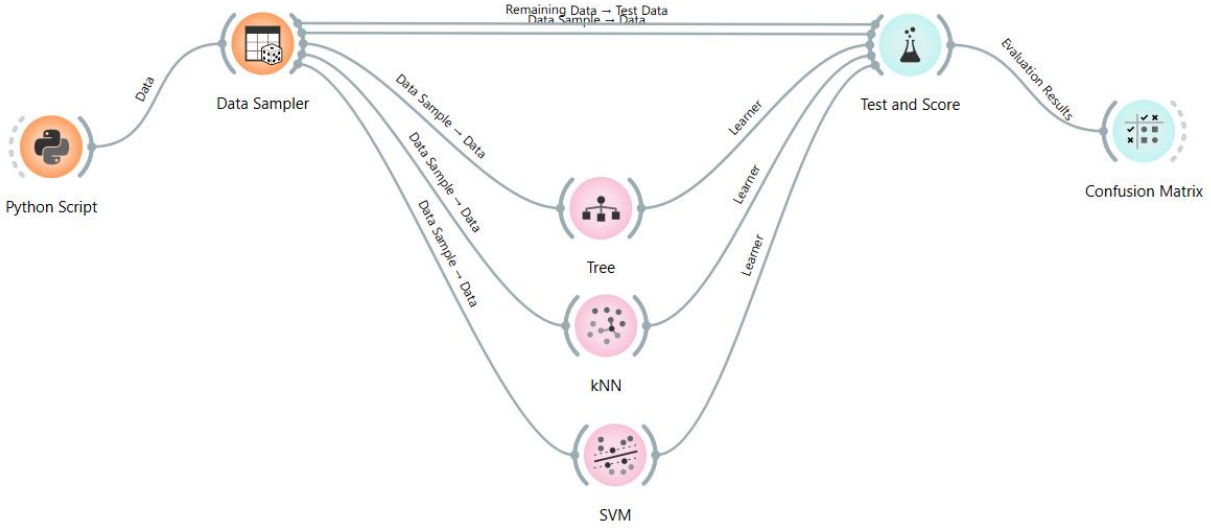
           0     1.0000    1.0000    1.0000        36
           1     0.9474    1.0000    0.9730        36
           2     1.0000    1.0000    1.0000        35
           3     1.0000    1.0000    1.0000        37
           4     1.0000    1.0000    1.0000        36
           5     1.0000    1.0000    1.0000        37
           6     1.0000    1.0000    1.0000        36
           7     0.9730    1.0000    0.9863        36
           8     1.0000    0.9429    0.9706        35
           9     1.0000    0.9722    0.9859        36

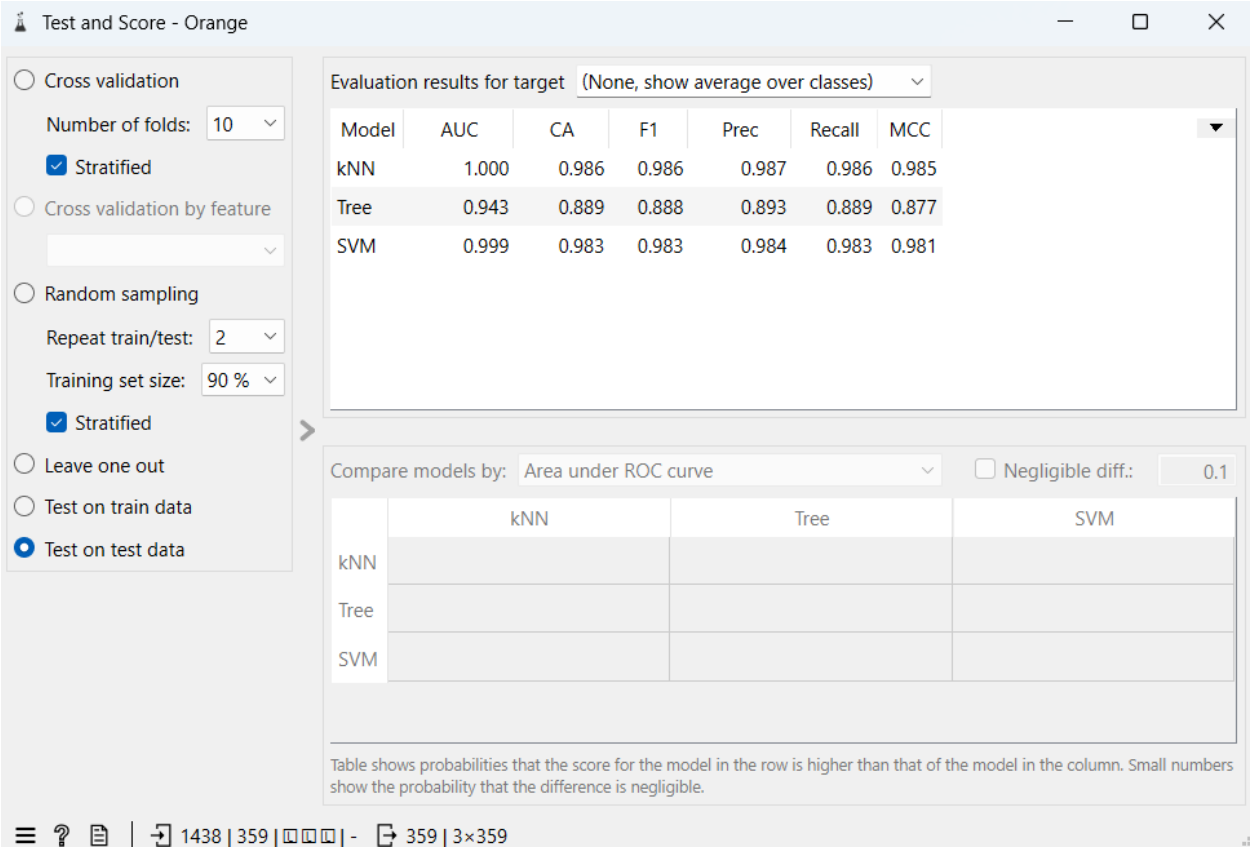
    accuracy                         0.9917       360
   macro avg     0.9920    0.9915    0.9916       360
weighted avg     0.9920    0.9917    0.9917       360

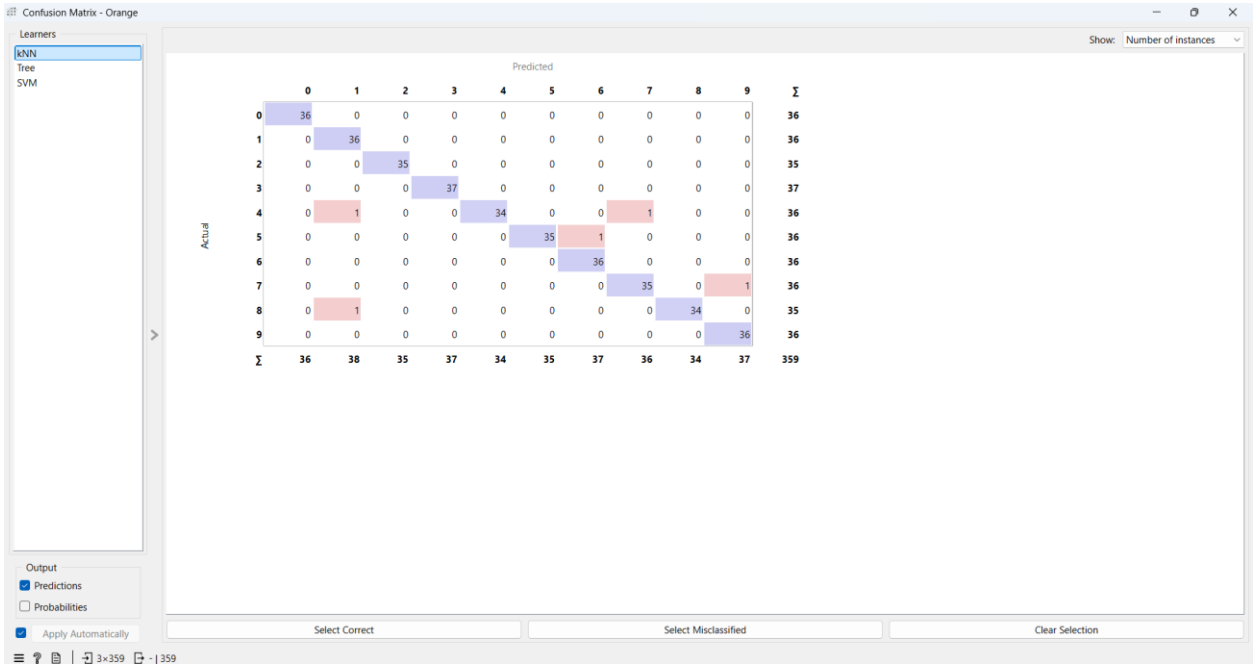

Сравнение моделей:
k-NN -> 0.9861
Decision Tree -> 0.825
SVM -> 0.9917

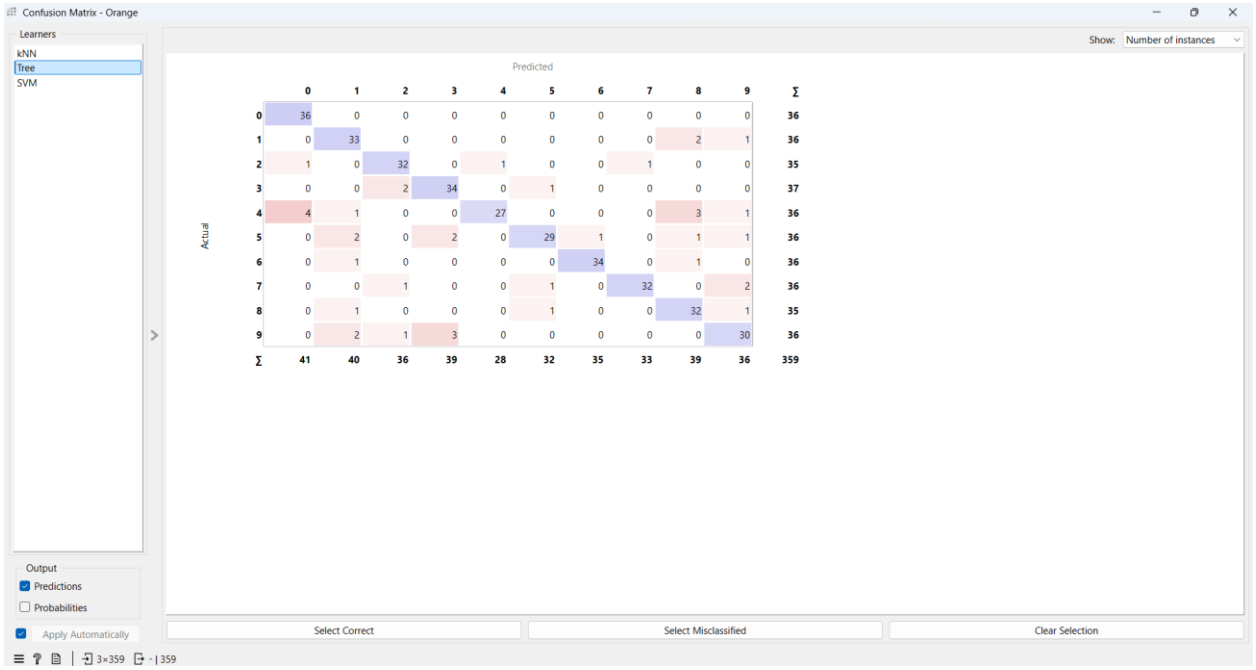
Лучшая модель: SVM с точностью 0.9917
```

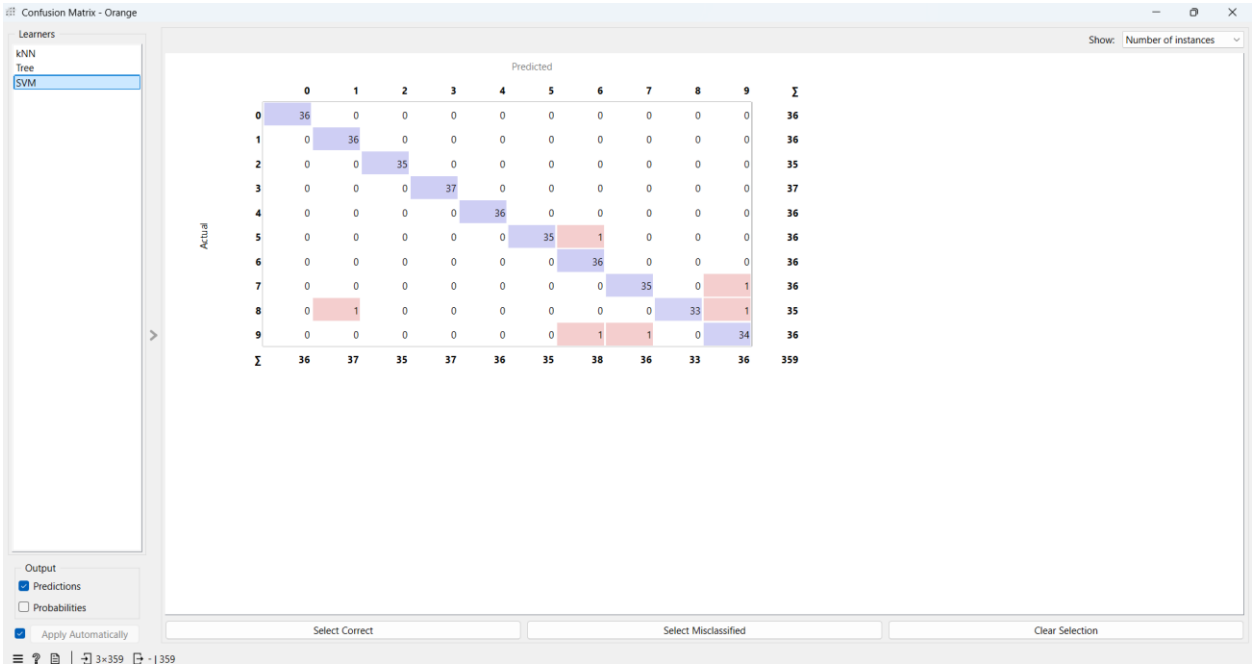
Результаты программы Orange:






Confusion Matrix - Orange
Learners
kNN
Tree
SVM
Show:
Number of instances
Predicted
Actual
Output
Predictions
Probabilities
Apply Automatically
Select Correct
Select Misclassified
Clear Selection
3×359
359


Confusion Matrix - Orange
Learners
kNN
Tree
SVM
Show:
Number of instances
Predicted
Actual
Output
Predictions
Probabilities
Apply Automatically
Select Correct
Select Misclassified
Clear Selection
3×359
359



Вывод:

В ходе лабораторной работы был исследован датасет Digits (распознавание рукописных цифр 0–9). Данные были разделены на обучающую (80%) и тестовую (20%) выборки. Были обучены три модели: k-ближайших соседей (k-NN), дерево решений и метод опорных векторов (SVM).